C# PRACTICALS

Practical 1a and Practical 1b

Practical 1a: Floyd's Triangle till N Rows in C#

Code:

```
using System;
namespace prac1a
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Floyds triangle");
            Console.Write("Enter the number of rows:");
            int n = int.Parse(Console.ReadLine());
            int num = 1;

            for (int i = 1; i <= n; i++)
            {
                for (int j = 1; j <= i; j++)
                {
                    Console.Write(num + " ");
                    num++;
                }
                Console.WriteLine();
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Floyds triangle
Enter the number of rows:5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Practical 1b: Arrange Names of Cities in Sorted Order

Code:

```
using System;
namespace SortCities
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] cities = new string[10];

            Console.WriteLine("=== City Name Sorter ===");
            Console.WriteLine("Enter names of 10 cities:");

            for (int i = 0; i < 10; i++)
            {
                cities[i] = Console.ReadLine();
            }

            // Sort array using Array.Sort()
            Array.Sort(cities);
            Console.WriteLine("\n--- Cities in Sorted Order ---");

            foreach (string i in cities)
            {
                Console.WriteLine(i);
            }
        }
    }
}
```

Output:

```
=== City Name Sorter ===
Enter names of 10 cities:
delhi
chennai
surat
jaipur
mumbai
pune
bangalore
hyderabad
kolkata
ahmedabad

--- Cities in Sorted Order ---
ahmedabad
bangalore
chennai
delhi
hyderabad
jaipur
kolkata
mumbai
pune
surat
```

C# PRACTICAL 2

Function Overloading, Inheritance, Interface and Exception Handling

Practical 2A: Function Overloading

Code:

```
using System;

namespace B4_Arjun_Nitin
{
    class area
    {
        public void CalculateArea(int side)
        {
            Console.WriteLine("Area of Square = " + (side * side));
        }

        public void CalculateArea(int length, int breadth)
        {
            Console.WriteLine("Area of Rectangle =" + (breadth * length));
        }

        public double CalculateArea(float p, double rad)
        {
            return (p * rad * rad);
        }
    }

    internal class functionoverloading
    {
        static void Main(string[] args)
        {
            area obj = new area();

            Console.WriteLine("Enter the side");
            int side = Convert.ToInt32(Console.ReadLine());
            obj.CalculateArea(side);

            Console.WriteLine("Enter the length of Rectangle :");
            int length = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("Enter the breadth of Rectangle :");
            int breadth = Convert.ToInt32(Console.ReadLine());
            obj.CalculateArea(length, breadth);

            Console.WriteLine("Enter the Radius of circle :");
            double rad = double.Parse(Console.ReadLine());
            Console.WriteLine("circle" + obj.CalculateArea(3.14f, rad));
        }
    }
}
```

Output:

```
Enter the side
2
Area of Square = 4
Enter the length of Rectangle :
3
Enter the breadth of Rectangle :
2
Area of Rectangle =6
Enter the Radius of circle :
5
circle78.5000026226044
```

Practical 2B: Inheritance

Code:

```
using System;

namespace B4_Arjun_Nitin
{
    class vehicle
    {
        string name = "Audi";

        public void dispvehicle()
        {
            Console.WriteLine("vehicle name =" + name);
        }
    }

    class car : vehicle
    {
        string model = "2026";

        public void dispcar()
        {
            dispvehicle();
            Console.WriteLine("Car model =" + model);
        }
    }

    class sportscar : car
    {
        int speed = 450;

        public void dispsportscar()
        {
            dispcar();
            Console.WriteLine("Max speed =" + speed);
        }
    }

    class bike : vehicle
    {
        string type = "classic";

        public void dispbike()
        {
            dispvehicle();
            Console.WriteLine("bike type =" + type);
        }
    }

    internal class inheritance
    {
        static void Main(string[] args)
        {
            sportscar s = new sportscar();
            bike b = new bike();

            s.dispsportscar();
            b.dispbike();
        }
    }
}
```

Output:

```
vehicle name =Audi
Car model =2026
Max speed =450
vehicle name =Audi
bike type =classic
```

Practical 2C: Interface

Code:

```
using System;

interface Area
{
    double compute(double x);
}

class square : Area
{
    public double compute(double x)
    {
        return x * x;
    }
}

class circle : Area
{
    public double compute(double x)
    {
        return Math.PI * x * x;
    }
}

class interf
{
    public static void Main(string[] args)
    {
        square sq = new square();
        circle c = new circle();
        Area a;

        a = sq as Area;
        Console.WriteLine("Area of Square=" + a.compute(10.0));

        a = c as Area;
        Console.WriteLine("Area of Circle=" + a.compute(10.0));
    }
}
```

Output:

```
Area of Square=100
Area of Circle=314.159265358979
```

Practical 2D: Exception Handling

Code:

```
using System;

class invalidage : Exception
{
    public invalidage(String msg) : base(msg)
    {
    }
}

class excepthan
{
    static void ValidateAge(int age)
    {
        if (age < 0 || age > 100)
        {
            throw new invalidage("invalid Age");
        }
    }

    public static void Main(string[] args)
    {
        try
        {
            int a = 10, b = 2;
            Console.WriteLine(a / b);
        }
        catch (DivideByZeroException)
        {
            Console.Write("Deniminator should not be zero");
        }
        finally
        {
            Console.Write("In Finally....");
        }

        try
        {
            Console.Write("\nEnter age:");
            int age = int.Parse(Console.ReadLine());
            ValidateAge(age);
        }
        catch (invalidage ex)
        {
            Console.WriteLine("Try with other Input !");
        }
    }
}
```

Output (valid age):

```
5
In Finally....
Enter age:35
```

Output (invalid age):

```
5
In Finally....
Enter age:109
Try with other Input !
```

C# PRACTICAL 3

Delegates, Multicast Delegate and BankAccount Property Validation

Practical 3a: Arithmetic Methods via C# Delegates

Code:

```
using System;

namespace caldelegate
{
    delegate double ArithmeticOperation(double a, double b);

    class Calculator
    {
        public static double Add(double a, double b)
        {
            return a + b;
        }

        public static double Subtract(double a, double b)
        {
            return a - b;
        }

        public static double Multiply(double a, double b)
        {
            return a * b;
        }

        public static double Divide(double a, double b)
        {
            if (b == 0)
            {
                Console.WriteLine("Division by zero is not possible.");
                return 0;
            }
            return a / b;
        }

        static void Main(string[] args)
        {
            Console.Write("Enter First Number: ");
            double num1 = Convert.ToDouble(Console.ReadLine());

            Console.Write("Enter Second Number: ");
            double num2 = Convert.ToDouble(Console.ReadLine());

            ArithmeticOperation op;

            op = Add;
            Console.WriteLine("Addition = " + op(num1, num2));

            op = Subtract;
            Console.WriteLine("Subtraction = " + op(num1, num2));

            op = Multiply;
            Console.WriteLine("Multiplication = " + op(num1, num2));

            op = Divide;
            Console.WriteLine("Division = " + op(num1, num2));
        }
    }
}
```

Output:

```
Enter First Number: 300
Enter Second Number: 200
Addition = 500
Subtraction = 100
Multiplication = 60000
Division = 1.5
```

Practical 3b: Multicast Delegate

Code:

```
using System;

namespace multicast_delegate
{
    delegate void Mdelegate();

    class DM
    {
        public static void display()
        {
            Console.WriteLine("hello");
        }

        public static void Print()
        {
            Console.WriteLine("hello world");
        }
    }

    class Mtest
    {
        public static void Main()
        {
            Mdelegate m1 = new Mdelegate(DM.display);
            Mdelegate m2 = new Mdelegate(DM.Print);
            Mdelegate m3 = m1 + m2;
            Mdelegate m4 = m2 + m1;
            Mdelegate m5 = m3 - m2;

            Console.WriteLine("delegate M3:");
            m3();

            Console.WriteLine("delegate M4:");
            m4();

            Console.WriteLine("delegate M5:");
            m5();
        }
    }
}
```

Output:

```
Multicast delegate
delegate M3:
hello
hello world
delegate M4:
hello world
hello
delegate M5:
hello
```

Practical 3c: BankAccount Class with Property Validation

Code:

```
using System;

namespace bankaccount
{
    class BankAccount
    {
        // Private field for account number
        private string accountnumber;

        // Property for AccountNumber
        public string AccountNumber
        {
            get { return accountnumber; }
            set { accountnumber = value; }
        }

        // Private field for balance
        private double balance;

        // Property with validation
        public double Balance
        {
            get { return balance; }
            set
            {
                if (value >= 0)
                {
                    balance = value;
                }
                else
                {
                    Console.WriteLine("Balance cannot be negative.");
                }
            }
        }

        // Constructor
        public BankAccount(string accountNumber, double balance)
        {
            AccountNumber = accountNumber;
            Balance = balance;
        }

        // Method to display account details
        public void Display()
        {
            Console.WriteLine("\nAccount Details");
            Console.WriteLine("Account Number: " + AccountNumber);
            Console.WriteLine("Balance: " + Balance);
        }
    }

    class Program
    {
        static void Main()
        {
            Console.Write("Enter Account Number: ");
            string accNo = Console.ReadLine();

            Console.Write("Enter Balance: ");
            double bal = Convert.ToDouble(Console.ReadLine());

            BankAccount account = new BankAccount(accNo, bal);
            account.Display();

            Console.ReadKey();
        }
    }
}
```

Output:

```
Enter Account Number: 123456789
Enter Balance: 5000

Account Details
```

```
Account Number: 123456789
Balance: 5000
```

PRACTICAL 10 (A)

Create a PWA with a Web App Manifest and implement Push Notifications

Aim

To create a Progressive Web Application (PWA) that includes an app name, icons, start URL, display mode, and push-notification functionality.

1. Create the Project

1. Open Visual Studio 2026.
2. Select Create a new project.
3. Choose ASP.NET Core Empty Web App.
4. Click Next.
5. Enter the project name: PushNotification.
6. Click Create.

2. Create the Manifest File

1. In Solution Explorer, right-click the project and select Add → New Folder.
2. Name the folder wwwroot (if it does not already exist).
3. Right-click wwwroot → Add → New Item → JSON File.
4. Name the file manifest.json.
5. Enter the following JSON and save the file:

```
{
  "name": "My PWA Application",
  "short_name": "MyPWA",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

3. Add Icons

1. Right-click wwwroot → Add → New Folder.
2. Name the folder icons.
3. Add icon1.png with dimensions 192 × 192 pixels.
4. Add icon2.png with dimensions 512 × 512 pixels.
5. Ensure the file names and paths match those specified in manifest.json.

4. Create the HTML Page

1. Right-click wwwroot → Add → New Item → HTML Page.
2. Name the file index.html.
3. Add the manifest link and app.js script as shown below, then save the file:

```
<!DOCTYPE html>
<html>
<head>
    <title>My PWA</title>
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#0d6efd">
</head>
<body>
    <h1>My Progressive Web App</h1>
    <button onclick="showNotification()">Show Notification</button>
    <script src="app.js"></script>
</body>
</html>
```

5. Create the JavaScript File for Notifications

1. Right-click wwwroot → Add → New Item → JavaScript File.
2. Name the file app.js.
3. Enter the following code and save it:

```
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}

function showNotification() {
    Notification.requestPermission().then(permission => {
        if (permission === 'granted') {
            navigator.serviceWorker.ready.then(registration => {
                registration.showNotification('PWA Notification', {
                    body: 'Push Notification Working Successfully!',
                    icon: 'icons/icon1.png'
                });
            });
        }
    });
}
```

6. Create the Service Worker

1. Right-click wwwroot → Add → New Item → JavaScript File.
2. Name the file service-worker.js.
3. Enter the following code and save it:

```
self.addEventListener('install', event => {
    console.log('Service Worker Installed');
});

self.addEventListener('activate', event => {
    console.log('Service Worker Activated');
});
```

7. Configure Program.cs

Open Program.cs and ensure static files are enabled. The file should contain the following basic configuration:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Note: If the project template already includes any of these lines, avoid adding duplicates.

8. Run the Application

1. Run the project from Visual Studio.
2. Open the local HTTPS address shown by Visual Studio, for example https://localhost:7236.
3. The page should display “My Progressive Web App” and a “Show Notification” button.
4. If the install icon is available in the browser address bar, the PWA can be installed.

9. Install the PWA

1. Open the website in Google Chrome.
2. Click the Install icon in the address bar (if shown).
3. Confirm installation. The application should open like a standalone app.

10. Test Push Notification

1. Click the Show Notification button.
2. When the browser asks for notification permission, click Allow.
3. A notification should appear with the title “PWA Notification” and the message “Push Notification Working Successfully!”.

Expected Result

A PWA page opens in the browser, can be installed when supported, and displays a notification after the user grants permission.

Important Note

This practical demonstrates a local browser notification through the Service Worker. It does not implement server-based Web Push subscriptions or VAPID keys. Notifications generally require HTTPS or localhost, and the icon files must exist at the specified paths.

Practical 4

4a: Student Registration Form

STEPS

1. Open Visual Studio → File → New → Project → ASP.NET Web Application (.NET Framework) → Click Next.
2. Project name → Click Create.
3. Choose Empty → Add folders & core reference, choose Web Forms → Click Create.
4. Add Empty Web Form → Solution Explorer: Right-click Project Name → Add → Web Form.
5. Enter WebForm name → Click Add.
6. Open WebForm1.aspx and add the following markup.
7. Open WebForm1.aspx.cs and add the code-behind logic.

WebForm1.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm1.aspx.cs" Inherits="WebApplication1.WebForm1" %>
<!DOCTYPE html>
<html>
<form id="form1" runat="server">
<div><h2>Student Registration Form</h2><hr /></div>
<asp:Label Text="Student Name:" runat="server" />
<asp:TextBox ID="txtName" runat="server" Width="200px" /> <br />
<asp:Label Text="Gender:" runat="server" /><br />
<asp:RadioButton ID="rbMale" GroupName="gender" Text="Male" runat="server" />
<asp:RadioButton ID="rbFemale" GroupName="gender" Text="Female" runat="server" /><br />
<asp:Label Text="Course:" runat="server" />
<asp:DropDownList ID="ddlCourse" runat="server">
<asp:ListItem Text="Select Course" Value="" />
<asp:ListItem Text="BSc IT" Value="BSc IT" />
<asp:ListItem Text="BSc CS" Value="BSc CS" />
<asp:ListItem Text="BCA" Value="BCA" />
</asp:DropDownList><br />
<asp:Label Text="Date of Birth:" runat="server" />
<asp:Calendar ID="calDOB" runat="server" /><br />
<asp:CheckBox ID="chkTerms" runat="server" Text="I accept Terms and Conditions" /><br />
<asp:Button ID="btnSubmit" runat="server" OnClick="btnSubmit_Click" Text="submit" />
  
<asp:Button ID="btnClear" runat="server" Text="Clear" />
<br /><br />
<asp:Label ID="lblOutput" runat="server" ForeColor="Green" Font-Bold="true" />
</form>
</html>
```

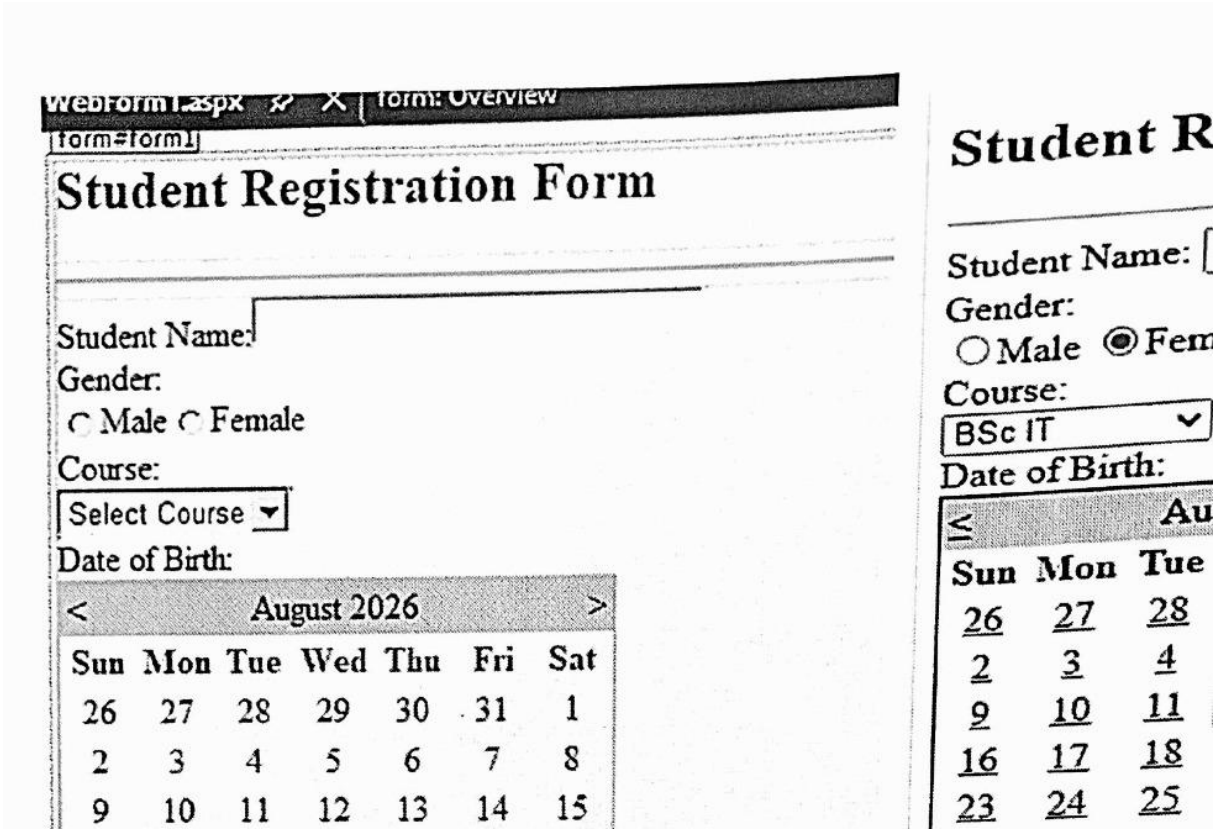
WebForm1.aspx.cs

```
using System;
namespace WebApplication1
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e) { }
        protected void btnSubmit_Click(object sender, EventArgs e)
        {
            string gender = "";
            if (rbMale.Checked) { gender = rbMale.Text; }
            else if (rbFemale.Checked) { gender = rbFemale.Text; }

            if (chkTerms.Checked)
```

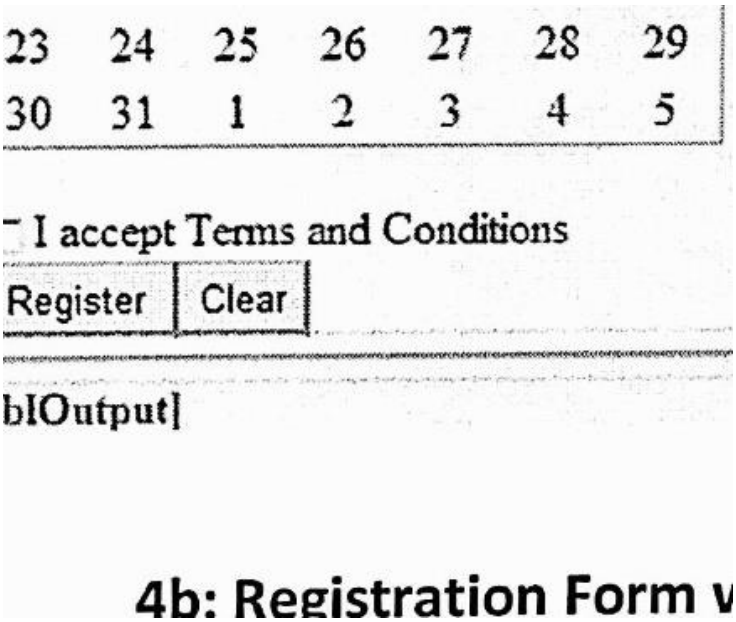
```
            {
                lblOutput.Text = "Student Name: " + txtName.Text + "<br/>" +
                    gender + "<br/>Course: " + ddlCourse.SelectedValue +
                    "<br/>Date: " + calDOB.SelectedDate.ToString();
            }
            else { lblOutput.Text = "please accept terms & condition"; }
        }
    }
}
```

Output / Diagram



4b: Registration Form with Validation Controls

Output / Diagram



WebForm1.aspx

```
<form id="form1" runat="server">
<h2>Registration with Validation</h2>
Name:
<asp:TextBox ID="txtName" runat="server" />
<asp:RequiredFieldValidator ControlToValidate="txtName" ErrorMessage="Name is required!" ForeColor="Red"
runat="server" />
<br />
Age:
<asp:TextBox ID="txtAge" runat="server" />
<asp:RequiredFieldValidator ControlToValidate="txtAge" ErrorMessage="Age required!" ForeColor="Red" runat="server"
/>
<asp:RangeValidator ControlToValidate="txtAge" Type="Integer" MinimumValue="18" MaximumValue="60"
ErrorMessage="Age must be 18-60" ForeColor="Red" runat="server" />
<br />
Email:
<asp:TextBox ID="txtEmail" runat="server" />
<asp:RegularExpressionValidator ControlToValidate="txtEmail" ValidationExpression="^[\w\.\-]+@([\w\-]+\.)+[\w\-
]{2,4}$" ErrorMessage="Invalid Email!" ForeColor="Red" runat="server" />
<br />
Password:
<asp:TextBox ID="txtPass" TextMode="Password" runat="server" />
<asp:RequiredFieldValidator ControlToValidate="txtPass" ErrorMessage="Password required!" ForeColor="Red"
runat="server" />
<br />
Confirm:
<asp:TextBox ID="txtConfirm" TextMode="Password" runat="server" />
<asp:CompareValidator ControlToValidate="txtConfirm" ControlToCompare="txtPass" ErrorMessage="Passwords do not
match!" ForeColor="Red" runat="server" />
<br />
<asp:Button ID="btnReg" Text="Register" runat="server" OnClick="btnReg_Click" Height="31px" />
<asp:ValidationSummary runat="server" ForeColor="Red" />
<asp:Label ID="lblMsg" runat="server" ForeColor="Green" />
</form>
```

WebForm1.aspx.cs

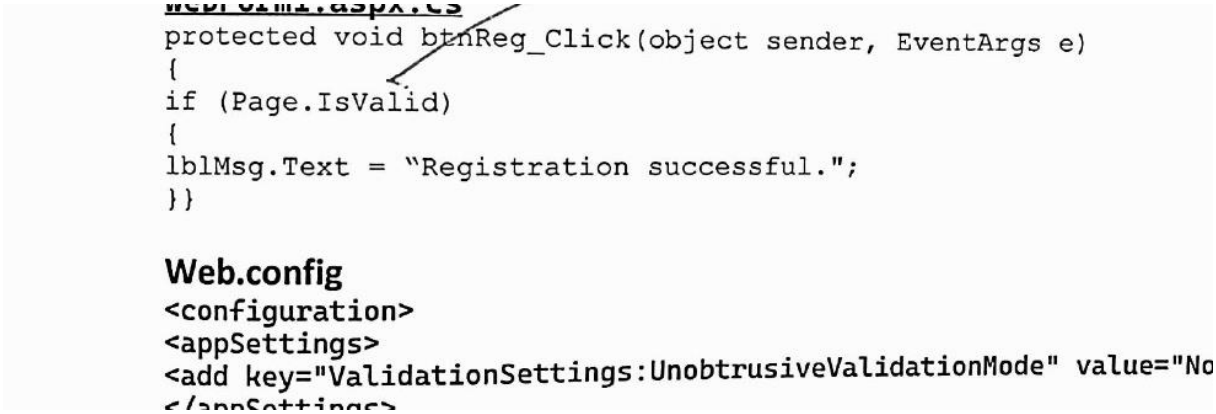
```
protected void btnReg_Click(object sender, EventArgs e)
```

```
{
    if (Page.IsValid)
    {
        lblMsg.Text = "Registration successful.";
    }
}
```

Web.config

```
<configuration>
  <appSettings>
    <add key="ValidationSettings:UnobtrusiveValidationMode" value="None" />
  </appSettings>
  <system.web>
    <compilation debug="true" targetFramework="4.7.2" />
    <httpRuntime targetFramework="4.7.2" />
  </system.web>
</configuration>
```

Validation Output / Diagram



4c: AdRotator Control

XMLFile.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<Advertisements>
  <Ad>
    <ImageUrl>image1.jpg</ImageUrl>
    <NavigateUrl>Default3.aspx</NavigateUrl>
    <AlternateText>advertisement</AlternateText>
    <Impressions>1</Impressions>
    <Keyword>food</Keyword>
  </Ad>
  <Ad>
    <ImageUrl>image2.jpg</ImageUrl>
    <NavigateUrl>Default3.aspx</NavigateUrl>
    <AlternateText>advertisement</AlternateText>
    <Impressions>1</Impressions>
    <Keyword>food</Keyword>
  </Ad>
  <Ad>
    <ImageUrl>image3.jpg</ImageUrl>
    <NavigateUrl>https://thebelgianwaffle.co/</NavigateUrl>
    <AlternateText>advertisement</AlternateText>
    <Impressions>1</Impressions>
    <Keyword>food</Keyword>
  </Ad>
  <Ad>
    <ImageUrl>image4.jpg</ImageUrl>
    <NavigateUrl>https://mcdelivery.co.in/</NavigateUrl>
    <AlternateText>advertisement</AlternateText>
    <Impressions>1</Impressions>
    <Keyword>coffee</Keyword>
  </Ad>
  <Ad>
    <ImageUrl>image5.jpg</ImageUrl>
    <NavigateUrl>https://www.zomato.com/mumbai/desi-chai-theka-4-vasai</NavigateUrl>
    <AlternateText>advertisement</AlternateText>
    <Impressions>1</Impressions>
    <Keyword>coffee</Keyword>
  </Ad>
</Advertisements>
```

Default.aspx

```
<form id="form1" runat="server">
<h2>AdRotator Control Demo</h2>
<p>Refresh the page to see a different advertisement:</p>
<div>
<asp:AdRotator ID="AdRotator1" runat="server" AdvertisementFile="~/XMLFile.xml"
    Height="60px" Width="468px" />
<br />
<asp:Button ID="Button1" runat="server" Text="Show Another Ad" />
</div>
</form>
```

AdRotator Output / Diagram



Practical 5A: Create an MVC Application to Perform Basic Arithmetic Operations

Steps

1. Open Visual Studio 2022/2026.

2. Create a new project.

- Click “Create a new project”.
- Search for MVC.
- Select “ASP.NET Core Web App (Model-View-Controller)”.
- Click Next.

2.2 Enter the project name as arithmeticmvc, click Next, and select the latest available framework.

3. Create Model File.

- In Solution Explorer (if it is not visible, open View → Solution Explorer).
- Right-click Models → Add → Class.
- Enter the file name Calculator.cs and click Add.

Calculator.cs — Model Code

```
namespace arithmeticmvc.Models
{
    public class Calculator
    {
        public double Number1 { get; set; }
        public double Number2 { get; set; }
        public required string operation { get; set; }
        public double Result { get; set; }
    }
}
```

Press Ctrl + S to save the file.

4. Open and Modify the Controller

In Solution Explorer, open Controllers → HomeController.cs. Replace the existing controller code with the following:

HomeController.cs

```
using arithmeticmvc.Models;
using Microsoft.AspNetCore.Mvc;
```

```
using System.Diagnostics;

namespace arithmeticmvc.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index(Calculator calc)
        {
            switch (calc.operation)
            {
                case "+":
                    calc.Result = calc.Number1 + calc.Number2;
                    break;

                case "-":
                    calc.Result = calc.Number1 - calc.Number2;
                    break;

                case "*":
                    calc.Result = calc.Number1 * calc.Number2;
                    break;

                case "/":
                    if (calc.Number2 != 0)
                    {
                        calc.Result = calc.Number1 / calc.Number2;
                    }
                    break;
            }

            return View(calc);
        }
    }
}
```

Save the file using Ctrl + S.

5. Open the View Folder

In Solution Explorer, open Views → Home and double-click Index.cshtml. Delete the existing code and paste the following view code.

Index.cshtml — View Code

```
@model arithmeticmvc.Models.Calculator

<h2>Arithmetic operator</h2>

<form method="post">
    <label>Enter number 1</label>
```

```
    <input type="number" step="any" asp-for="Number1" />
    <br /><br />

    <label>Enter Number 2</label>
    <input type="number" step="any" asp-for="Number2" />
    <br /><br />

    <label>Select operation</label>
    <select asp-for="operation">
        <option value="+">Addition</option>
        <option value="-">Subtraction</option>
        <option value="*">Multiplication</option>
        <option value="/">Division</option>
    </select>

    <br /><br />
    <input type="submit" value="Calculate" />
</form>

@if (Model != null)
{
    <h3>Result = @Model.Result</h3>
}
```

6. Run the Application

1. Save all files.
2. Click the Run button or press F5.
3. Enter the first and second numbers.
4. Select Addition, Subtraction, Multiplication, or Division.
5. Click Calculate to display the result.

Expected Output

The page displays an arithmetic calculator with two number fields, an operation dropdown, and a Calculate button. After calculation, the result appears below the form.

Original Practical Screenshot

Practical 5A: Create an MVC Application to perform basic Arithmetic Operations.

Steps

1: Open visual studio 2022/2026

2: Create new project

- Click on create a new project
- Search for MVC
- Select project type as ASP.NET Core Web App(Model-View-controller)
- Then click next

2.2 Name the project arithmeticmvc->Click Next->Select Framework (latest available)

3. **Create Model File**

- In solution Explorer (If this window is not seen, go to view Menu and select it.)
- Right-click: Models →Add→Class
- Enter File name as Calculator.cs and click Add

```
namespace arithmeticmvc.Models{
public class Calculator{
public double Number1{ get;set;}
public double Number2{ get;set;}
public required string operation { get; set; }
public double Result { get; set; }
```

Press Ctrl+ S to save

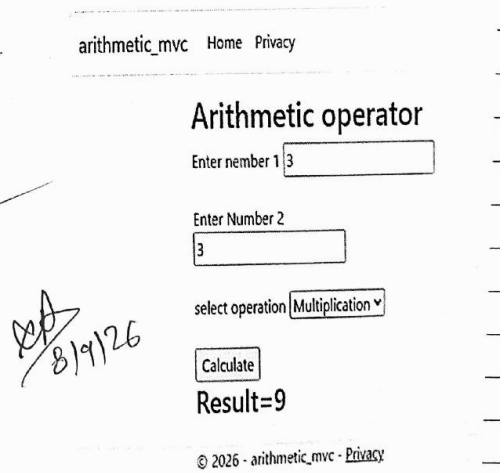
4. Open Controller

In solution explorer: open Controllers => Double click HomeController.cs

Modify Controller code:

Remove existing code inside HomeController and replace with:

```
using arithmeticmvc.Models;
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;
namespace arithmeticmvc.Controllers{
public class HomeController : Controller
{ public IActionResult Index(Calculator calc){
switch (calc.operation){
case "+":
calc.Result = calc.Number1 + calc.Number2;
break;
case "-":
calc.Result = calc.Number1 - calc.Number2;
break;
case "*":
calc.Result = calc.Number1 * calc.Number2;
break;
case "/":
if (calc.Number2 != 0)
{
calc.Result = calc.Number1 / calc.Number2;
}
break;} return View(calc);}}}
```



Save the file

5. Open views Folder

In solution explorer: Open: => Views -> Home

Double-click :=> Index.cshtml

11: Replace Existing View Code: Delete all existing code and paste:

Additional View Code (as shown on the second image)

The second image continues the Index.cshtml view code for Practical 5A. The code is included above in a clean, editable format. The original second page also begins Practical 5B, which has been excluded as requested.

Practical 5B: Different Types of Cookies

1. Create Project

Step 1: Open Visual Studio 2026.

Step 2: Create an ASP.NET Web Application (.NET Framework).

Step 3: Select “Empty” and check “Web Forms”.

Step 4: Click Create.

2. Add Web Form

Right-click Project → Add → Web Form.

Name: CookieDemo.aspx

3. Add Controls

Add the following controls to the Web Form:

Control Label Text	Control Type	ID
User Name	TextBox	txtName
Save Cookie	Button	btnSave
Show Cookies	Button	btnShow
Output	Label	lblOutput

4. Write Code for Save Cookie Button

```
protected void btnSave_Click(object sender, EventArgs e)
{
    HttpCookie sessionCookie = new HttpCookie("SessionUser");
    sessionCookie.Value = txtName.Text;
    Response.Cookies.Add(sessionCookie);

    HttpCookie persistentCookie = new HttpCookie("PersistentUser");
    persistentCookie.Value = txtName.Text;
    persistentCookie.Expires = DateTime.Now.AddDays(7);
    Response.Cookies.Add(persistentCookie);

    lblOutput.Text = "Cookies Saved Successfully";
}
```

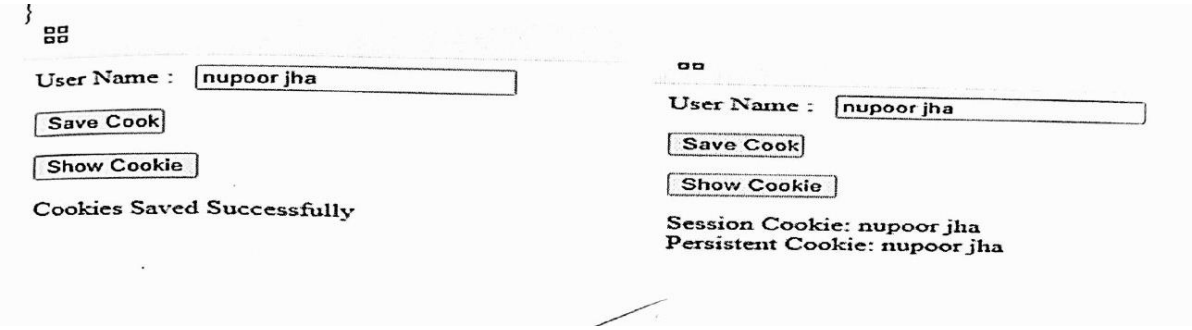
5. Write Code for Show Cookie Button

```
protected void btnShow_Click(object sender, EventArgs e)
{
    string sessionData = Request.Cookies["SessionUser"].Value;
    string persistentData = Request.Cookies["PersistentUser"].Value;

    lblOutput.Text = "Session Cookie: " + sessionData +
                     "<br/>Persistent Cookie: " + persistentData;
}
```

6. Output

The screenshots below are kept as original images from the provided practical pages.



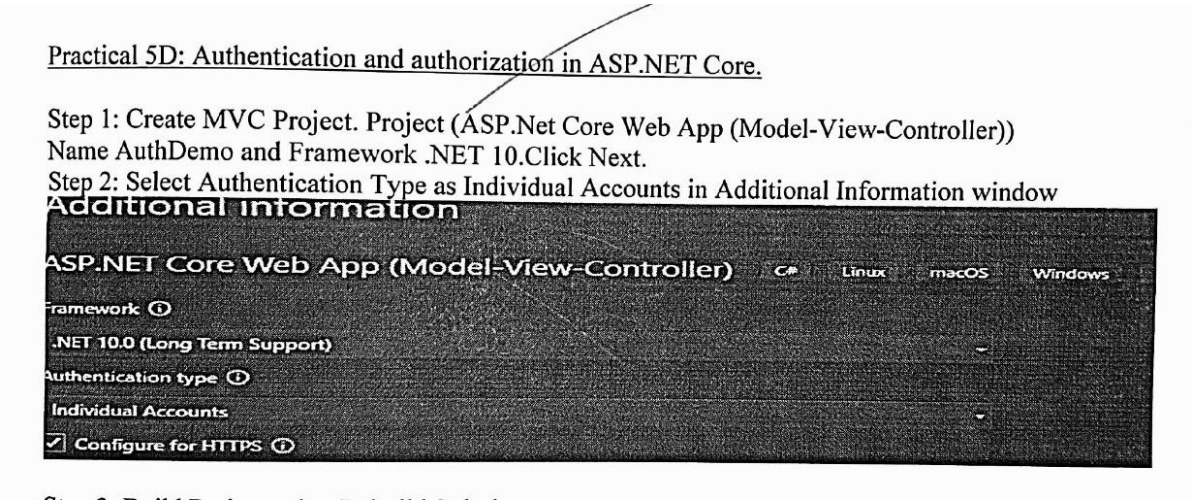
Practical 5C: Authentication and Authorization in ASP.NET Core

Step 1: Create an MVC Project.

Select ASP.NET Core Web App (Model-View-Controller).

Project Name: AuthDemo. Select the available .NET framework and click Next.

Step 2: Select Authentication Type as Individual Accounts in the Additional Information window.



Step 3: Build the project using Rebuild Solution.

Go to Build → Rebuild Solution.

Step 4: Create Database

Open Tools → NuGet Package Manager → Package Manager Console.

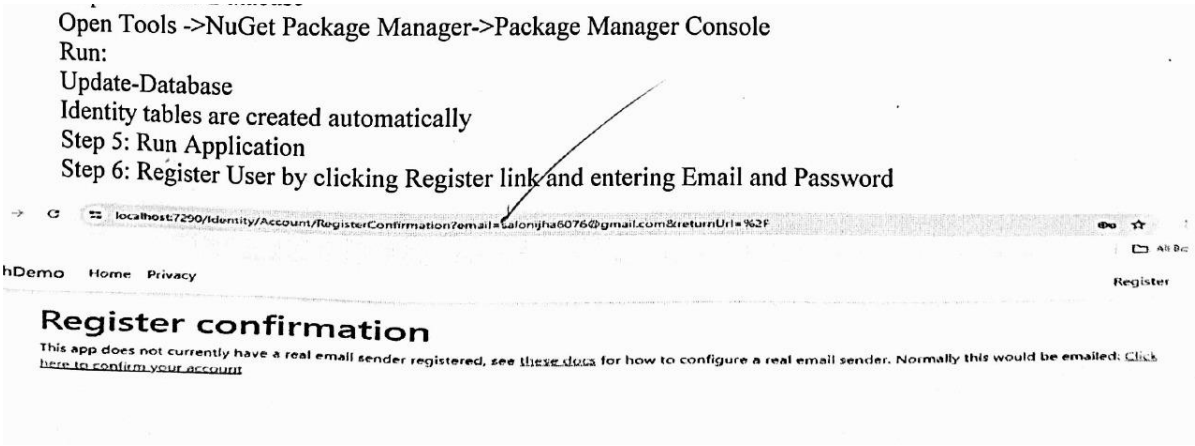
Run the following command:

```
Update-Database
```

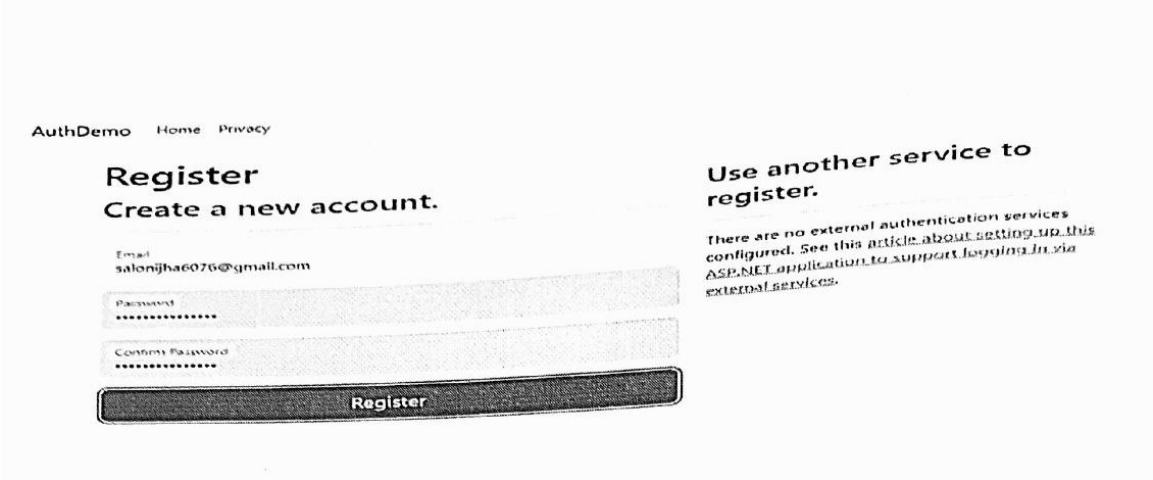
Identity tables are created automatically.

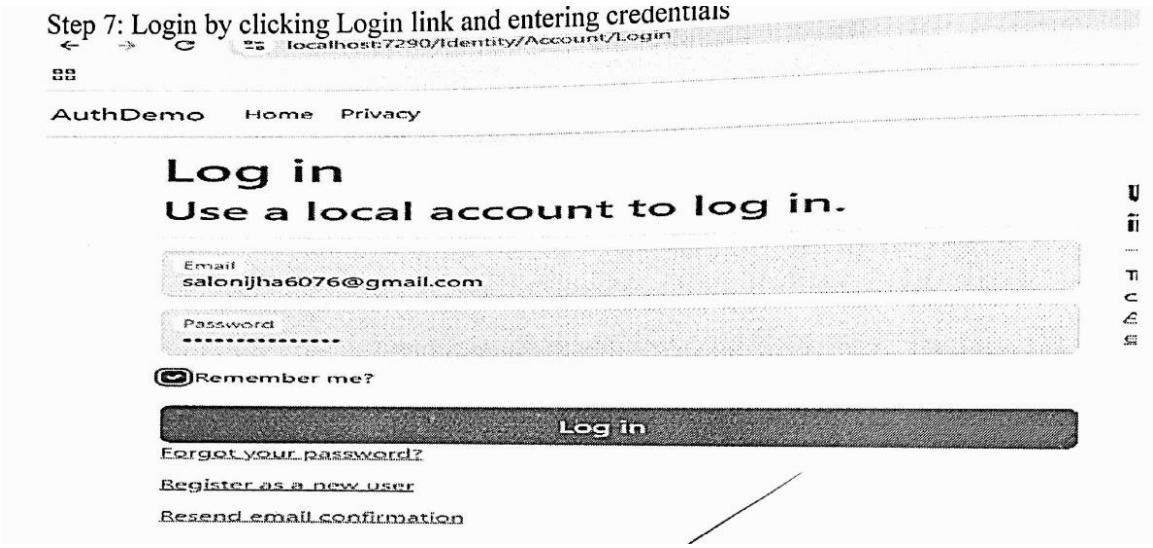
Step 5: Run Application.

Step 6: Register a user by clicking the Register link and entering an email address and password.



Step 7: Log in by clicking the Login link and entering the registered credentials.





Step 8: Add Authorized Page

Create a new method in HomeController and apply the Authorize attribute.

Add the namespace:

```
using Microsoft.AspNetCore.Authorization;
```

Add the following method:

```
[Authorize]
public IActionResult Secret()
{
    return View();
}
```

Step 9: Create Secret View in Views/Home Folder

Create the folder only if needed. Right-click the Home folder → Add → View → Select Razor View - Empty.

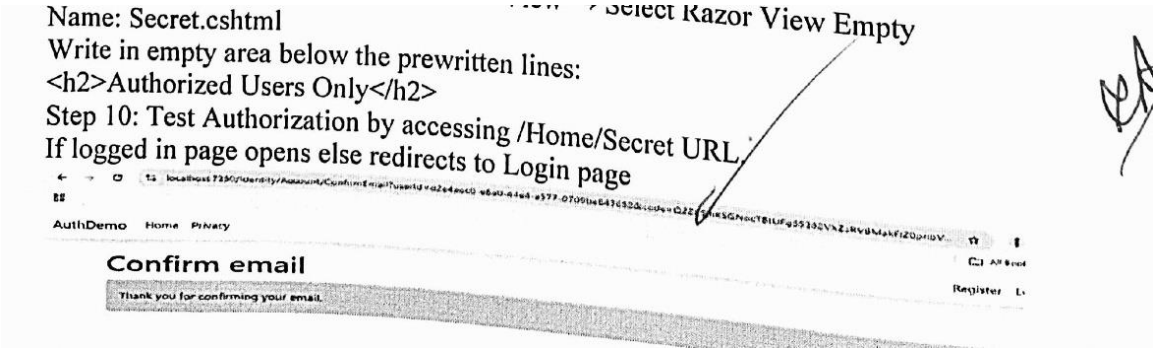
Name: Secret.cshtml

Write the following line in the empty area below the prewritten lines:

```
<h2>Authorized Users Only</h2>
```

Step 10: Test Authorization

Access the /Home/Secret URL. If you are logged in, the page opens; otherwise, you are redirected to the Login page.



Practical 5D

Develop a Web Application to Perform CRUD Operations on an Employee Table Using EF Core

Aim

To create an ASP.NET Core MVC application that performs Create, Read, Update, and Delete (CRUD) operations on employee records using Entity Framework Core and SQL Server.

Procedure

1. Create a New MVC Project

1. Open Visual Studio 2026.
2. Select Create a new project.
3. Choose ASP.NET Core Web App (Model-View-Controller), then click Next.
4. Set Project name to EmployeeCRUD and choose a suitable location.
5. Select .NET 10.0 (or the latest framework available in your Visual Studio installation).
6. Click Create.

2. Install Entity Framework Core Packages

Open Tools → NuGet Package Manager → Manage NuGet Packages for Solution. Install compatible versions of the following packages. Use the same version for all EF Core packages; the reference practical specifies version 10.0.8 or a compatible version available in your environment.

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.Tools
- Microsoft.EntityFrameworkCore.Design
- Microsoft.VisualStudio.Web.CodeGeneration.Design

3. Create the Employee Model

In Solution Explorer, right-click Models → Add → Class. Name the file Employee.cs and enter:

```
namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }

        public required string Name { get; set; }

        public required string Department { get; set; }

        public double Salary { get; set; }
    }
}
```

4. Create the Data Folder

Right-click the project → Add → New Folder. Name the folder Data.

5. Create the DbContext Class

Right-click the Data folder → Add → Class. Name the file AppDbContext.cs and enter:

```
using EmployeeCRUD.Models;
using Microsoft.EntityFrameworkCore;

namespace EmployeeCRUD.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(
            DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

6. Add the Database Connection String

Open appsettings.json. Add a ConnectionStrings section. The LocalDB connection below follows the reference practical; ensure SQL Server LocalDB is installed. If you use another SQL Server instance, change the Server value accordingly.

```
{
  "ConnectionStrings": {
    "MyConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCert
ificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

7. Configure DbContext in Program.cs

Add the required using statements at the top of Program.cs. Register AppDbContext before builder.Build(), as shown:

```
using EmployeeCRUD.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
```

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

8. Create the Employee Controller Using Scaffolding

1. Build the project first.
2. In Solution Explorer, right-click Controllers → Add → New Scaffolded Item.
3. Select MVC Controller with views, using Entity Framework.
4. Click Add.
5. Select Model class: Employee (EmployeeCRUD.Models).
6. Select Data context class: AppDbContext (EmployeeCRUD.Data).
7. Keep the generated controller name as EmployeesController, then click Add.

Visual Studio generates the controller and views for the standard CRUD operations: Index, Details, Create, Edit, and Delete.

9. Create the Database

Open Tools → NuGet Package Manager → Package Manager Console. Run the following commands one at a time:

```
Add-Migration InitialCreate
Update-Database
```

If the migration reports a package advisory or warning, first check whether it is only a warning or an actual build/migration error. Update packages to compatible versions where possible, then rebuild the project and run the migration commands again. Avoid adding an advisory-suppression entry unless your instructor or project administrator specifically requires it.

10. Run and Test the Application

1. Build the solution: Build → Build Solution.
2. Run the project using HTTPS or IIS Express.
3. In the browser address bar, append /Employees to the application URL. For example: https://localhost:7000/Employees (use the port shown on your own machine).
4. Test the following operations:

- Create: add an employee with name, department, and salary.
- Read: verify that the employee appears in the list and open Details.
- Update: select Edit, change employee information, and save.
- Delete: select Delete and confirm removal.

Expected Result

The EmployeeCRUD application runs successfully and displays an employee list. Users can create, view, edit, and delete employee records stored in the EmployeeDB database through Entity Framework Core.

Important Notes

- Make sure the project builds successfully before running migrations.
- Use a connection string that matches the SQL Server instance installed on your computer.
- If your installed .NET version differs from .NET 10, choose a framework supported by your Visual Studio installation and use compatible EF Core package versions.
- The generated EmployeesController and Razor views provide the CRUD interface.

Web Technology Practicals – Practical 6A & 6B

Combined practical instructions transcribed from the provided images.

Practical 6A: College Information Portal as a Progressive Web Application (PWA)

Objective: Create a simple website for a college information portal and convert it into a progressive web application.

1. Create an ASP.NET Core Web App.

Search for “ASP.NET Core Web App (Model-View-Controller)” and click Next.

Project Name: CollegePortalPWA

Framework: .NET 10 or the latest available version. Click Create.

2. Update the Home Page.

Open Views → Home → Index.cshtml and replace the existing HTML with:

```
<div class="text-center">
    <h1>Vartak College Information Portal</h1>
    <h3>Welcome to Vartak College</h3>
    <p>Courses Offered:</p>
    <ul>
        <li>BSC IT</li>
        <li>BSC CS</li>
        <li>BSC DS</li>
    </ul>
    <p>Contact: 9818763456</p>
</div>
```

3. Add PWA support – Service Worker.

Right-click wwwroot → Add → New Item → JavaScript File. Name it service-worker.js.

```
self.addEventListener('install', function (event) {
    console.log('Service Worker Installed');
});
self.addEventListener('fetch', function (event) {
    console.log('Fetching:', event.request.url);
});
```

4. Create the Manifest File.

Right-click wwwroot → Add → New Item → JSON File. Name it manifest.json.

```
{
  "name": "Vartak College Information Portal",
  "short_name": "CollegePortal",
  "start_url": "/",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
```

```
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

5. Create an icons folder.

Right-click wwwroot → Add → New Folder. Name it icons. Add icon1.png and icon2.png inside it.

6. Register the manifest.

Open Views → Shared → _Layout.cshtml. Add this tag inside the <head> section:

```
<link rel="manifest" href="~/manifest.json">
```

7. Register the Service Worker.

In the same _Layout.cshtml file, add the following script before </body>:

```
<script>
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js')
        .then((registration) => {
            console.log('Service Worker Registered!', registration);
        })
        .catch((error) => {
            console.log('Service Worker Registration Failed:', error);
        });
}
</script>
```

8. Verify that static files are enabled.

In Program.cs, ensure this line exists; otherwise, add it in the appropriate place:

```
app.UseStaticFiles();
```

9. Run the application.

Open the application in Chrome and click the PWA Install icon in the address bar to install the app.

10. Verify the Service Worker.

Press F12 → Application → Service Workers. Check that service-worker.js is activated and running.

Practical 6B: Create a PWA that Displays a Custom Offline Page

Objective: Create a PWA that displays a custom offline page when the user is not connected to the internet.

1. Create an ASP.NET Core MVC project.

Project Name: OfflinePWA

2. Select the framework.

Choose .NET 10 or the latest available version and click Create.

3. Create the Home Page.

Open Views → Home → Index.cshtml and add:

```
<div class="text-center">
    <h1>VARTAK College Information Portal</h1>
    <h3>Welcome to VARTAK College</h3>
    <p>This website supports Offline Mode.</p>
</div>
```

4. Create the Offline Page.

Right-click wwwroot → Add → New Item → HTML Page. Name it offline.html.

```
<!DOCTYPE html>
<html>
<head>
    <title>Offline</title>
</head>
<body>
    <h1>No Internet Connection</h1>
    <h3>Please check your internet connection.</h3>
</body>
</html>
```

5. Create the Manifest File.

Right-click wwwroot → Add → New Item → JSON File. Name it manifest.json.

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

6. Create the icons folder.

Right-click wwwroot → Add → New Folder. Name it icons. Add icon1.png and icon2.png inside it.

7. Create the Service Worker for offline caching.

Right-click wwwroot → Add → New Item → JavaScript File. Name it service-worker.js.

```
const CACHE_NAME = "offline-cache-v1";
const urlsToCache = ["/", "/offline.html"];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME).then(cache => {
            return cache.addAll(urlsToCache);
        })
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request).catch(() => {
            return caches.match("/offline.html");
        })
    );
});
```

8. Register the Manifest.

Open Views → Shared → _Layout.cshtml and add this link tag inside <head>:

```
<link rel="manifest" href="~/manifest.json">
```

9. Register the Service Worker.

In the same _Layout.cshtml file, add the following script before </body>:

```
<script>
if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js')
        .then(() => console.log('Service Worker Registered'));
}
</script>
```

10. Verify static files.

In Program.cs, ensure the following line exists:

```
app.UseStaticFiles();
```

11. Run the application.

12. Verify the Service Worker.

Press F12 → Application → Service Workers and verify the status.

13. Test the Offline Page.

Press F12 → Network tab → select Offline → refresh the page.

Note: The source images show the practical instructions. Icon image files (icon1.png and icon2.png) must be supplied or created separately for the manifest icons to display correctly.

Practical 7A

Aim: Develop a PWA that works in offline mode using Service Worker.

Step 1: Create ASP.NET Core MVC Project

1. Open Visual Studio 2026.

2. Click Create a new project.

3. Search for ASP.NET Core Web App (Model-View-Controller).

4. Select the project template and click Next.

5. Enter Project Name: SWOfflinePWA.

6. Select Framework: .NET 10, then click Create.

Step 2: Create Home Page

1. Open Views → Home → Index.cshtml.

2. Replace the existing code with:

```
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>PWA Offline Demo</h3>
    <p>This application works without internet connection.</p>
</div>
```

3. Save the file.

Step 3: Create Manifest File

1. Right-click wwwroot → Add → New Item.

2. Select JSON File and name it manifest.json.

3. Enter the following code:

```
{
  "name": "Offline PWA",
  "short_name": "OfflinePWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

4. Save the file.

Step 4: Create Icons Folder

1. Right-click wwwroot → Add → New Folder.

2. Name the folder icons.

3. Add icon1.png (192 × 192 pixels) and icon2.png (512 × 512 pixels) inside it.

Step 5: Create Service Worker

1. Right-click wwwroot → Add → New Item → JavaScript File.

2. Name the file service-worker.js.

3. Enter the following code:

```
const CACHE_NAME = "pwa-cache-v1";
const urlsToCache = ["/", "/Home/Index"];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => cache.addAll(urlsToCache))
    );
});

self.addEventListener("fetch", event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => response || fetch(event.request))
    );
});
```

4. Save the file.

Step 6: Register Manifest

1. Open Views → Shared → _Layout.cshtml.

2. Add this line inside the <head> section:

```
<link rel="manifest" href="/manifest.json">
```

3. Save the file.

Step 7: Register Service Worker

1. In the same _Layout.cshtml file, add the following script before </body>:

```
<script>
    if ('serviceWorker' in navigator) {
        navigator.serviceWorker.register('/service-worker.js')
            .then(() => console.log('Service Worker Registered'));
    }
</script>
```

2. Save the file.

3. Open Program.cs and ensure app.UseStaticFiles(); exists before app.MapControllerRoute(...). Add it if missing.

Step 8: Run Application

1. Run the project in Visual Studio.

2. Open the application in Google Chrome.

3. Check whether the PWA installation icon appears in the browser.

4. Press F12 to open Developer Tools.

Step 9: Verify Service Worker

1. Open the Application tab in Developer Tools.

2. Select Service Workers from the left panel.

3. Check that service-worker.js is registered and its status is activated and running.

Step 10: Test Offline Mode

1. Keep the application open in Chrome.

2. Press F12 and select the Network tab.

3. Enable the Offline option.

4. Refresh the webpage.

5. Verify that the cached webpage opens without an internet connection.

Result

A Progressive Web Application (PWA) was developed using ASP.NET Core MVC and a Service Worker. The application was tested in offline mode.

Note: Serve the application over HTTPS or localhost. The pages must be successfully cached before going offline. If caching fails, check that the listed routes return successful responses.

Practical 8A

Aim: Create a PWA application with an HTML page that acts as the starting point of the application. The HTML page contains a link to manifest.json.

Step 1: Create a Web Project

1. Open Visual Studio 2026 and click Create a new project.

2. Select ASP.NET Core Web App (Model-View-Controller), then click Next.

Step 2: Configure the Project

1. Project Name: BasicPWA

2. Framework: .NET 10 or the latest available version.

3. Click Create.

Step 3: Enable Static Files

1. Open Program.cs.

2. Configure the application to serve the default HTML page and static files. Use the following minimal setup:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();

app.Run();
```

Note: If you keep the MVC template's existing middleware and routes, ensure UseDefaultFiles() and UseStaticFiles() are included before app.Run().

Step 4: Create the Starting HTML Page

1. Right-click wwwroot → Add → New Item → HTML Page.

2. Name the file index.html.

3. Enter the following code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Basic PWA</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="manifest" href="manifest.json">
</head>
<body>
    <h1>My Progressive Web Application</h1>
    <p>This is the starting page of PWA.</p>
</body>
</html>
```

4. Save the file.

Step 5: Link the Manifest File in HTML

The link to manifest.json is already included inside the <head> section of index.html:

```
<link rel="manifest" href="manifest.json">
```

Step 6: Create Icons Folder

1. Right-click wwwroot → Add → New Folder.

2. Name the folder icons.

3. Add the following PNG files to the icons folder:

   • icon1.png — 192 × 192 pixels

   • icon2.png — 512 × 512 pixels

Step 7: Create Manifest File

1. Right-click wwwroot → Add → New Item → JSON File.

2. Name the file manifest.json.

3. Enter the following JSON:

```
{
  "name": "Basic Progressive Web Application",
  "short_name": "BasicPWA",
  "description": "Simple Example",
  "start_url": "/index.html",
  "display": "standalone",
  "background_color": "white",
  "theme_color": "blue",
  "orientation": "portrait",
  "icons": [
    {
      "src": "/icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

4. Save the file.

Step 8: Run the Application

1. Run the project in Visual Studio.

2. Open the browser URL and navigate to /index.html if it does not open automatically.

3. Verify that the page displays “My Progressive Web Application” and “This is the starting page of PWA.”

4. Check the browser address bar for the PWA installation option. The app must be served through HTTPS or localhost, and valid icon files must be present.

Result

A basic Progressive Web Application was created with an HTML starting page linked to a manifest.json file.

Practical 8B

Aim: Create a simple Web API that returns student data in JSON format.

Step 1: Create Web API Project

1. Open Visual Studio 2026.

2. Click Create a new project.

3. Select ASP.NET Core Web API and click Next.

Step 2: Configure Project

1. Project Name: StudentAPI

2. Framework: .NET 10 or the latest available version.

3. Click Create.

Step 3: Create Student Model

1. Right-click the project → Add → New Folder.

2. Name the folder Models.

3. Right-click Models → Add → Class.

4. Name the class Student.cs and enter:

```
namespace StudentAPI.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; } = "";
        public string Course { get; set; } = "";
        public int Age { get; set; }
    }
}
```

5. Save the file.

Step 4: Create API Controller

1. Right-click the project → Add → New Folder and name it Controllers, if it does not already exist.

2. Right-click Controllers → Add → Controller.

3. Choose API Controller - Empty.

4. Name it StudentController.cs.

Step 5: Write API Code

Open StudentController.cs and enter the following code:

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Models;

namespace StudentAPI.Controllers
{
    [ApiController]
```

```
    [Route("api/[controller]")]
    public class StudentController : ControllerBase
    {
        [HttpGet]
        public List<Student> GetStudents()
        {
            return new List<Student>
            {
                new Student
                {
                    Id = 1,
                    Name = "Rahul",
                    Course = "BSc IT",
                    Age = 20
                },
                new Student
                {
                    Id = 2,
                    Name = "Sneha",
                    Course = "BCA",
                    Age = 21
                }
            };
        }
    }
}
```

Save the file. Ensure Program.cs contains the controller configuration:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();

app.Run();
```

If the project already contains Swagger/OpenAPI setup, you can keep it; ensure AddControllers() and MapControllers() are present.

Step 6: Run the Application

1. Run the StudentAPI project in Visual Studio.

2. Note the HTTPS localhost address shown in the browser or launch profile.

Step 7: Test the API

1. Open a browser and enter the project URL followed by /api/student.

   Example: https://localhost:xxxx/api/student (replace xxxx with the actual port number).

2. The browser should display the student data in JSON format, including Rahul and Sneha.

Result

A simple ASP.NET Core Web API was created and tested successfully. It returns student data in JSON format.

PRACTICAL 9(A)

Aim: Create an API endpoint to calculate the factorial of a number and test it using Postman.

Steps / Algorithm

1. Create ASP.NET Core Web API Project

1. Open Visual Studio 2026.
2. Select Create a new project.
3. Choose ASP.NET Core Web API and click Next.

2. Configure Project

4. Project Name: FactorialAPI
5. Select Framework: .NET 8 or the latest available version.
6. Enable OpenAPI Support, if available, and click Create.

3. Create the Controller

7. In Solution Explorer, right-click the Controllers folder.
8. Select Add → Class and name the file MathController.cs.

4. Write the Factorial Logic

9. Replace the contents of MathController.cs with the following code:

```
using Microsoft.AspNetCore.Mvc;

namespace FactorialAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult GetFactorial(int number)
        {
            if (number < 0)
            {
                return BadRequest("Factorial not possible for negative numbers.");
            }

            long factorial = 1;

            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }

            return Ok(new
            {
                Number = number,
                Factorial = factorial
            });
        }
```

```
        }
    }
```

Note: This loop calculates factorials for non-negative integers. The long data type can overflow for large inputs; use small test values such as 0 to 20.

5. Run the Project

10. Press F5 or click Run to start the project.
11. Note the HTTPS localhost address shown in the browser or Visual Studio output, for example https://localhost:44390.

6. Test the API in a Browser

12. Open a browser and enter the URL below, replacing the port with your project's actual HTTPS port:

```
https://localhost:44390/api/math/factorial/5
```

13. The response should be JSON similar to:

```
{
  "number": 5,
  "factorial": 120
}
```

7. Test Using Postman

14. Open Postman and create a new HTTP request.
15. Select the GET method.
16. Enter the URL using your project's actual port:

```
https://localhost:44390/api/math/factorial/5
```

17. Click Send. The response should return status 200 OK and the factorial value 120.

8. Test a Negative Number

18. In Postman or the browser, enter the following URL, replacing the port if needed:

```
https://localhost:44390/api/math/factorial/-2
```

19. Send the request. The API should return a 400 Bad Request response with this message:

```
Factorial not possible for negative numbers.
```

Result

The ASP.NET Core Web API endpoint was created successfully to calculate the factorial of a number. The endpoint was tested in a browser and Postman for a valid number and a negative number.